

ImmoSN — Plateforme Immobilière Sénégalaise

Plateforme SaaS de gestion immobilière dédiée au marché sénégalais.

Elle permet la publication d'annonces, la prise de contact et de visite par des visiteurs non authentifiés (prospects), la conversion automatique en client à l'activation d'un contrat, la messagerie, la planification et le suivi des visites (affectation, rapport), la gestion des contrats (vente/location, résiliation, prolongation), le suivi des litiges, et l'envoi de notifications email/SMS.

Table des matières

- 1. [Vue d'ensemble](#)
- 2. [Stack technique](#)
- 3. [Architecture globale](#)
- 4. [Prérequis](#)
- 5. [Démarrage rapide \(développement\)](#)
- 6. [Démarrage en production](#)
- 7. [Variables d'environnement](#)
- 8. [Structure du projet](#)
- 9. [Modèles de données](#)
- 10. [API REST — Endpoints](#)
- 11. [Sécurité et authentification](#)
- 12. [Flux métier](#)
- 13. [Notifications email / SMS](#)
- 14. [Frontend — Vue.js](#)
- 15. [Géolocalisation](#)
- 16. [Déploiement Docker](#)
- 17. [Configuration Nginx](#)

1. Vue d'ensemble

ImmoSN est une application web full-stack organisée en deux services :

Service	Technologie	Rôle
Backend	Spring Boot 4.0.6 / Java 21	API REST, logique métier, sécurité, notifications
Frontend	Vue 3 / Vite	Interface utilisateur SPA
Base de données	PostgreSQL (Neon) / H2 (dev)	Persistance des données
Proxy	Nginx	Reverse proxy + serveur statique

Rôles et identités

Il n'existe **pas** de rôle **VISITEUR** en base. Un visiteur non authentifié est représenté par l'entité **Prospect** (sans compte **User**) : il peut contacter l'agence et demander une visite sans s'inscrire. Le prospect est

automatiquement converti en compte **CLIENT** lorsqu'un administrateur active le contrat qui lui est associé (voir §12 — [Conversion prospect → client](#)).

Rôle (RoleType)	Accès
(aucun — Prospect)	Consulter les annonces, contacter l'agence (/discussions/invite), demander une visite (/visites/invite), suivre sa demande par token (/visites/suivi/{token})
CLIENT	Navigation annonces, messagerie, visites, contrats, favoris, signalements
ADMIN	Gestion annonces, leads, visites (affectation, rapport), contrats, messages, dashboard
SUPER_ADMIN	Tout ADMIN + acceptation/refus des visites + gestion des comptes administrateurs

2. Stack technique

Backend

- **Framework** : Spring Boot 4.0.6
- **Langage** : Java 21
- **ORM** : Spring Data JPA / Hibernate
- **Sécurité** : Spring Security 6 + JWT (JJWT 0.11.5)
- **Validation** : Jakarta Bean Validation
- **Build** : Maven 3.9
- **Base de données** : PostgreSQL (prod, via Neon) / H2 (dev)
- **Scheduler** : Spring `@Scheduled` (expiration automatique des contrats)
- **Email** : `spring-boot-starter-mail` (SMTP, ex. Gmail)
- **SMS** : Twilio SDK
- **HTTP Client** : RestTemplate (géocodage Nominatim)

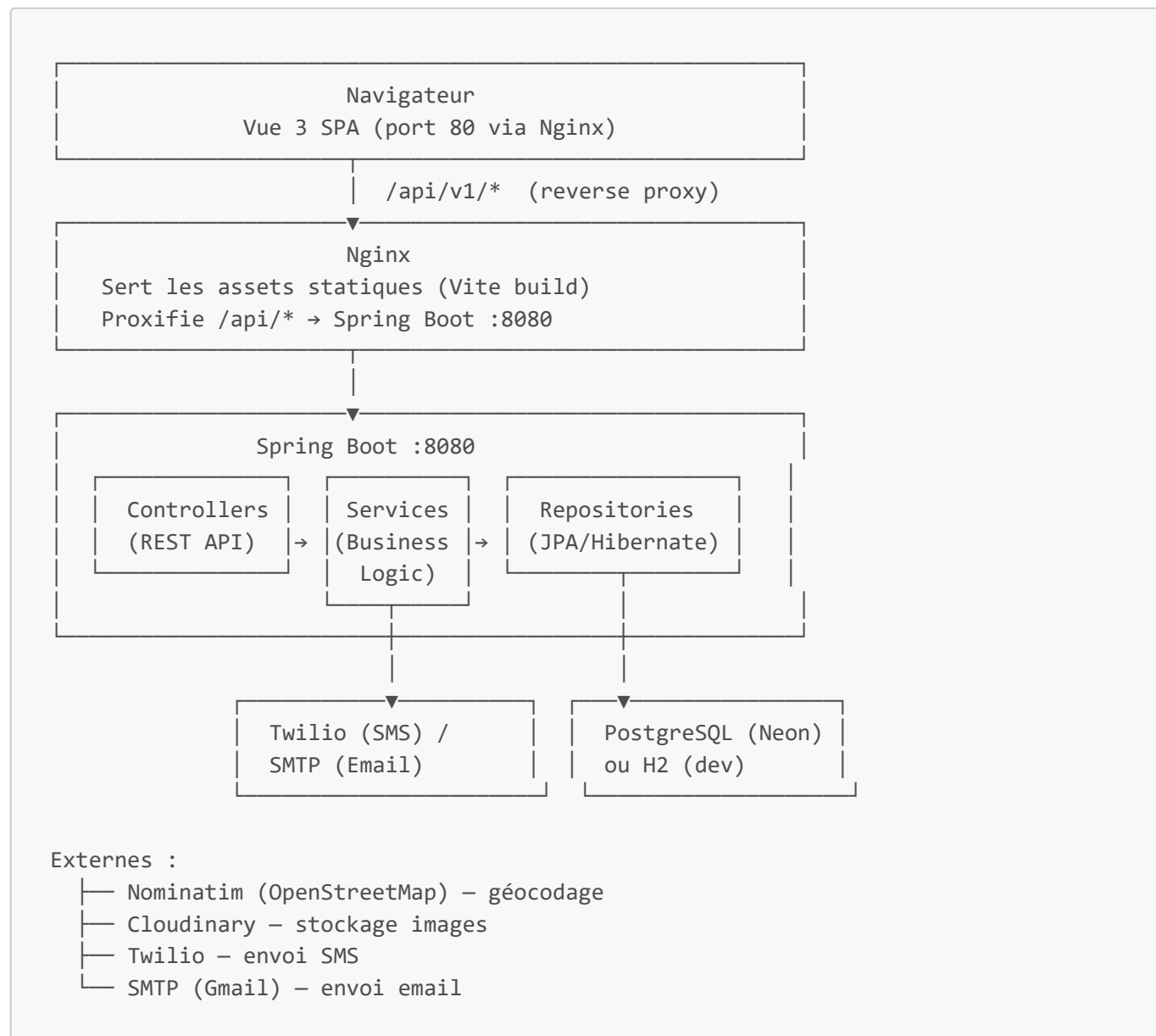
Frontend

- **Framework** : Vue 3 (Composition API, `<script setup>`)
- **Build** : Vite 8
- **Router** : Vue Router 5
- **State** : Pinia 3
- **HTTP** : Axios 1.16
- **Cartes** : Leaflet 1.9 + `@vue-leaflet/vue-leaflet`
- **Icônes** : `lucide-vue-next`
- **Images** : Cloudinary (upload direct)
- **Linting** : ESLint + oxlint + Prettier

Infrastructure

- **Conteneurisation** : Docker
- **Proxy** : Nginx (reverse proxy + SPA)
- **Orchestration** : Docker Compose
- **Stockage images** : Cloudinary CDN
- **Base prod** : PostgreSQL Neon (serverless, SSL)

3. Architecture globale



4. Prérequis

- **Docker** ≥ 24 et **Docker Compose** ≥ 2.20
- **Java 21** (développement backend sans Docker)
- **Node.js 20** (développement frontend sans Docker)
- **Maven 3.9** (build backend sans Docker)
- Compte **Cloudinary** (upload d'images)
- Compte **Neon** (base PostgreSQL en production)
- Compte **Twilio** (SMS — optionnel, désactivable)
- Compte SMTP (ex. Gmail avec mot de passe d'application — optionnel, désactivable)

5. Démarrage rapide (développement)

Avec Docker (recommandé)

```
# Cloner le projet
git clone <url-du-repo>
cd immosn

# Copier et remplir le fichier d'environnement
cp .env.example .env.dev
# Remplir .env.dev (voir section Variables d'environnement)

# Démarrer en mode développement (H2 in-memory)
docker compose -f docker-compose.dev.yml --env-file .env.dev up --build

# Application disponible sur :
# Frontend : http://localhost
# Backend  : http://localhost:8080
# H2 Console : http://localhost:8080/h2-console
```

Sans Docker — Backend seul

```
cd backend
# application.properties lit les variables d'environnement (voir §7) – les exporter
# ou les définir directement dans application.properties pour un essai local rapide

mvn spring-boot:run
# API disponible sur http://localhost:8080/api/v1
```

Sans Docker — Frontend seul

```
cd frontend
cp .env.example .env.local
# VITE_API_URL=http://localhost:8080/api/v1

npm install
npm run dev
# Application disponible sur http://localhost:5173
```

6. Démarrage en production

```
# Remplir le fichier .env avec les valeurs de production
cp .env.example .env
# Éditer .env (voir section Variables d'environnement)

# Déployer avec le compose de production (profil Spring "neon")
docker compose up -d --build

# Vérifier l'état des services
```

```
docker compose ps
docker compose logs -f backend
```

7. Variables d'environnement

Copier `.env.example` et remplir toutes les valeurs :

Base de données (PostgreSQL Neon)

Variable	Description	Exemple
DB_HOST	Hôte Neon (avec pooler)	ep-xxx-pooler.region.aws.neon.tech
DB_NAME	Nom de la base	neondb
DB_USER	Utilisateur PostgreSQL	neondb_owner
DB_PASSWORD	Mot de passe PostgreSQL	npg_xxxx
DB_POOL_SIZE	Taille du pool Hikari	5
DDL_AUTO	Stratégie JPA DDL (<code>update</code> en dev, <code>validate</code> en prod)	validate

Authentification JWT

Variable	Description	Valeur recommandée
JWT_SECRET	Clé secrète HMAC-SHA512 (min. 64 chars base64)	Générer : <code>openssl rand -base64 64</code>
JWT_EXPIRATION	Durée de vie du token (ms)	86400000 (24h)

Super Administrateur (bootstrap au démarrage)

Variable	Description
SUPER_ADMIN_EMAIL	Email du super admin créé automatiquement
SUPER_ADMIN_PASSWORD	Mot de passe (min. 8 chars, majuscule + chiffre + spécial)
SUPER_ADMIN_NOM	Nom complet
SUPER_ADMIN_TELEPHONE	Téléphone au format international

CORS et pagination

Variable	Défaut	Description
CORS_ORIGINS	<code>http://localhost,http://localhost:80</code>	Origines autorisées (séparées par virgule)

Variable	Défaut	Description
PAGE_DEFAULT_SIZE	10	Éléments par page (vue client)
PAGE_ADMIN_SIZE	20	Éléments par page (vue admin)
CONTRAT_EXPIRATION_CRON	0 5 0 * * *	Cron Spring d'expiration des contrats

Notifications — Twilio (SMS)

Variable	Défaut	Description
TWILIO_ENABLED	false	Active l'envoi SMS — laisser false si non configuré
TWILIO_ACCOUNT_SID	—	SID du compte Twilio
TWILIO_AUTH_TOKEN	—	Jeton d'authentification Twilio
TWILIO_FROM_NUMBER	—	Numéro expéditeur Twilio (format E.164)

Avec un compte Twilio **Trial**, seuls les numéros vérifiés dans la console Twilio (*Phone Numbers* → *Verified Caller IDs*) peuvent recevoir un SMS. Passer le compte en payant lève cette restriction.

Notifications — Email (SMTP)

Variable	Défaut	Description
MAIL_ENABLED	false	Active l'envoi email — laisser false si non configuré
MAIL_HOST	smtp.gmail.com	Hôte SMTP
MAIL_PORT	587	Port SMTP (STARTTLS)
MAIL_USERNAME	—	Identifiant SMTP
MAIL_PASSWORD	—	Mot de passe SMTP (mot de passe d'application pour Gmail)
MAIL_FROM	immosn.noreply@gmail.com	Adresse expéditrice

Cloudinary (upload images)

Variable	Description
VITE_CLOUDINARY_CLOUD_NAME	Nom du cloud Cloudinary
VITE_CLOUDINARY_UPLOAD_PRESET	Preset d'upload non signé

Frontend

Variable	Description	Exemple
VITE_API_URL	URL de base de l'API	/api/v1 (prod) ou http://localhost:8080/api/v1 (dev)

8. Structure du projet

```

immosn/
├── backend/                                # Spring Boot 4 / Java 21
│   ├── src/main/java/sn/immosn/backend/
│   │   ├── annonce/                      # Annonces immobilières
│   │   │   ├── data/entity/             # Annonce, TypeBienAnnonce, Commodity,
│   │   │   │   ├── TypeTransaction
│   │   │   │   ├── data/repository/
│   │   │   │   └── service/              # AnnonceService, CommodityService,
│   │   │   │   TypeBienService
│   │   │   │   ├── auth/                # Authentification & utilisateurs
│   │   │   │   └── data/entity/         # User, Role, RoleType, BlacklistedToken,
│   │   │   │   UserSession
│   │   │   │   ├── data/jwt/            # JwtTokenProvider, JwtAuthenticationFilter
│   │   │   │   ├── data/repository/
│   │   │   │   └── service/             # AuthService, RoleDataInitializer
│   │   │   │   prospect/               # Visiteurs non authentifiés
│   │   │   │   ├── data/entity/        # Prospect (token de suivi, convertedUser)
│   │   │   │   └── service/            # ProspectConversionService (conversion →
│   │   │   │   CLIENT)
│   │   │   │   ├── lead/                # Pipeline commercial
│   │   │   │   │   ├── data/entity/    # Lead, StatutLead
│   │   │   │   │   └── service/
│   │   │   │   ├── visite/             # Demandes de visite & rapports
│   │   │   │   │   ├── data/entity/    # DemandeVisite, StatutDemandeVisite,
│   │   │   │   │   └── service/        # DemandeVisiteService, RapportVisiteService,
│   │   │   │   RapportVisite
│   │   │   │   └── service/
│   │   │   │   VisiteHistoryService
│   │   │   │   ├── discussion/          # Messagerie client/prospect ↔ agence
│   │   │   │   │   ├── data/entity/    # Discussion, Message, SenderRole
│   │   │   │   │   └── service/
│   │   │   │   ├── contrat/            # Contrats de location/vente
│   │   │   │   │   ├── data/entity/    # Contrat, StatutContrat, TypeContrat
│   │   │   │   │   ├── scheduler/      # ContratExpirationJob (cron quotidien)
│   │   │   │   │   └── service/
│   │   │   │   ├── favoris/            # Annonces favorites
│   │   │   │   │   ├── data/entity/    # AnnonceFavoris (clé composite)
│   │   │   │   │   └── service/
│   │   │   │   ├── signalement/        # Litiges sur contrats
│   │   │   │   │   ├── data/entity/    # Signalement, StatutSignalement
│   │   │   │   │   └── service/
│   │   │   │   ├── dashboard/          # Statistiques admin
│   │   │   │   │   └── service/
│   │   │   │   ├── location/           # Géolocalisation (Nominatim)
│   │   │   │   │   ├── GeoCodingService.java
│   │   │   │   │   ├── LocationService.java
│   │   │   │   │   └── GeocodeResponseDto.java

```

```

| | | | client/web/ # Couche présentation (Controllers + DTOs)
| | | | | annonce/ # AnnonceController,
TypeBienAnnonceController, CommodityController
| | | | | auth/ # AuthController + DTOs + Mapper
| | | | | lead/ # LeadController + DTOs + Mapper
| | | | | visite/ # DemandeVisiteController (visites +
rapports) + DTOs + Mapper
| | | | | discussion/ # DiscussionController + DTOs + Mapper
| | | | | contrat/ # ContratController + DTOs + Mapper
| | | | | favoris/ # FavorisController + DTOs
| | | | | signalement/ # SignalementController + DTOs + Mapper
| | | | | location/ # LocationController
| | | | | dashboard/ # DashboardController + DTO
| | | | | config/
| | | | | | SecurityConfig.java # RBAC, CORS, JWT filter chain
| | | | | | RestTemplateConfig.java
| | | | | shared/
| | | | | | exception/ # GlobalExceptionHandler,
EntityNotFoundException, EntityExistException
| | | | | response/ # RestResponse<T>, PagedResponse<T>
| | | | | service/ # NotificationService,
VisiteTrackingNotificationService (email/SMS)
| | | | | Dockerfile # Multi-stage Maven build
| frontend/ # Vue 3 / Vite
| | src/
| | | main.js # Point d'entrée + Leaflet CSS
| | | App.vue # Shell SPA (RouterView)
| | | router/index.js # Vue Router + guards de navigation
| | | stores/
| | | | authStore.js # Auth Pinia (token, user, rôle)
| | | | favorisStore.js # Favoris Pinia (Set<id>, toggle optimiste)
| | | | toastStore.js # Toasts Pinia (success/error/warning)
| | | services/
| | | | api.js # Axios instance + intercepteurs JWT
| | | | authService.js
| | | | annonceService.js
| | | | leadService.js
| | | | visiteService.js
| | | | discussionService.js
| | | | contratService.js
| | | | favorisService.js
| | | | signalementService.js
| | | | dashboardService.js
| | | | locationService.js
| | | | cloudinaryService.js
| | | components/
| | | | AnnonceCard.vue # Carte annonce (avec toggle favori)
| | | | NavBar.vue
| | | | PhoneInput.vue # Saisie téléphone internationale (sélecteur
pays)
| | | | InputField.vue # Champ avec toggle afficher/masquer (mot de
passe)
| | | | LocationMap.vue # Carte Leaflet (lecture + draggable)

```



```

├── admin/                                # Composants admin réutilisables
├── layouts/
│   ├── ClientLayout.vue                 # Layout public (navbar + footer)
│   └── AdminLayout.vue                  # Layout admin (sidebar)
├── views/
│   ├── auth/                            # Inscription, Connexion, première connexion
│   │   (mdp temporaire)
│   ├── annonces/                       # Liste + Détail (public)
│   ├── profil/                         # Profil client
│   ├── discussions/                    # Messagerie client
│   └── visites/                        # Visites client + suivi invité + demande
├── invité dédiée
│   ├── contrats/                      # Contrats client
│   ├── signalements/                  # Litiges client
│   ├── favoris/                      # Favoris client
│   └── admin/                         # Toutes les vues admin
├── Dockerfile
├── nginx/
│   ├── nginx.conf                     # Reverse proxy (développement local)
│   └── nginx.vps.conf                 # Reverse proxy production (SSL/TLS, rate
limiting, HSTS)
├── Dockerfile                          # ARG NGINX_CONF – bascule auto vers
nginx.vps.conf en prod
├── docker-compose.yml                 # Production – VPS (Neon PostgreSQL,
NGINX_CONF=nginx.vps.conf)
├── docker-compose.dev.yml             # Développement (H2 in-memory)
├── .env.example                       # Template variables d'environnement
└── README.md

```

9. Modèles de données

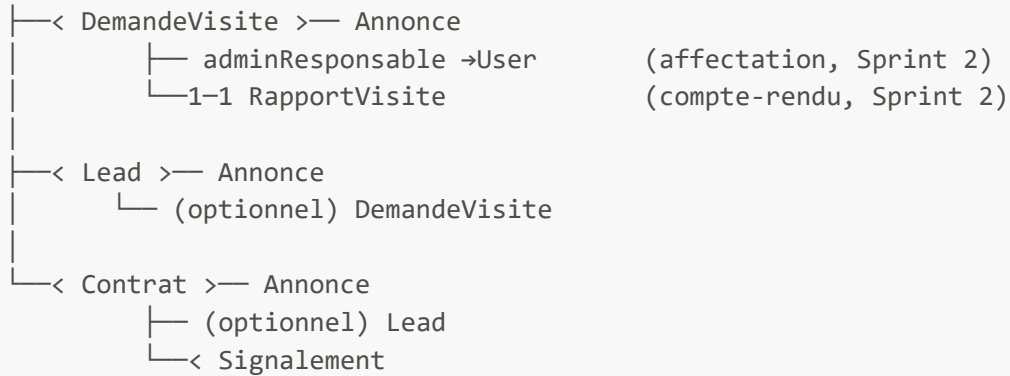
Diagramme des entités principales

```

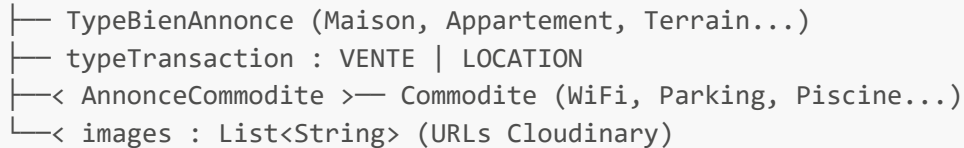
Prospect (visiteur non authentifié, token de suivi UUID)
├── < Discussion > — Annonce           (contact invité)
├── < DemandeVisite > — Annonce         (visite invité)
├── < Lead > — Annonce
├── < Contrat > — Annonce               (avant conversion)
└── convertedUser → User               (conversion automatique à l'activation du
contrat)

User (CLIENT / ADMIN / SUPER_ADMIN)
├── < AnnonceFavoris > — Annonce
├── < Discussion > — Annonce
└── < Message (SenderRole: CLIENT | ADMIN)

```



Annonce



Entités détaillées

User

id	Long	PK
nomComplet	String	NOT NULL
email	String	UNIQUE, NOT NULL
telephone	String	NOT NULL
adresse	String	nullable
photo	String	nullable (URL)
password	String	BCrypt
creationDate	LocalDateTime	@PrePersist
isArchived	boolean	default false
dernierConnexion	LocalDateTime	nullable
compteGenereAuto	boolean	default false – créé automatiquement via
conversion prospect		
motDePasseAChanger	boolean	default false – mot de passe temporaire à changer
à la 1re connexion		
roles	Set<Role>	ManyToMany (user_roles)

Prospect

id	Long	PK
nom	String	NOT NULL
prenom	String	nullable
email	String	NOT NULL
telephone	String	NOT NULL
adresse	String	nullable
token	String	UNIQUE, immutable – numéro de suivi UUID communiqué
au visiteur		
convertedUser	→User	nullable – rempli à la conversion, null tant que non

```

converti
createdAt      LocalDateTime @PrePersist
demandesVisite →[]DemandeVisite
discussions    →[]Discussion

```

Annonce

```

id          Long          PK
libelle     String        NOT NULL
description TEXT          nullable
nbrPieces   Integer       NOT NULL
surface     Double        NOT NULL
prix        BigDecimal    NOT NULL (precision 10, scale 2)
adresse     String        nullable
region      String        nullable (toujours "Dakar")
departement String        NOT NULL
quartier    String        NOT NULL
latitude    Double        nullable (géocodé par Nominatim)
longitude   Double        nullable (géocodé par Nominatim)
typeBien    →TypeBien    NOT NULL
typeTransaction VENTE | LOCATION – détermine le type de contrat créé à la clôture
de visite
annonceCommodites →[]      cascade
images       List<String> table annonce_images (URLs Cloudinary)
isArchived   boolean      default false
createdAt    LocalDateTime @PrePersist
updatedAt    LocalDateTime @PreUpdate

```

Lead

```

id          Long          PK
client      →User        nullable – client authentifié (exclusif avec prospect)
prospect    →Prospect     nullable – visiteur non authentifié (exclusif avec
client)
annonce     →Annonce      NOT NULL
visite      →DemandeVisite nullable
statut      StatutLead    EN_COURS | CONVERTI | ABANDONNE
noteAdmin   TEXT          nullable
convertedAt LocalDateTime nullable – horodatage EN_COURS → CONVERTI
createdAt / updatedAt

```

DemandeVisite

```

id          Long          PK
client      →User        nullable – visite d'un client authentifié
prospect    →Prospect     nullable – visite d'un visiteur non

```

authentifié		
adminResponsable	→User	nullable – administrateur affecté (Sprint 2)
annonce	→Annonce	NOT NULL
nom, prenom, telephone, email, adresse	String	nullable – identité figée du visiteur
invité		
dateVisite	LocalDateTime	NOT NULL
heureVisite (parcours invité)	String	nullable – heure saisie séparément
dateReplanificationProposee	LocalDateTime	nullable (Sprint 2)
statut	StatutDemandeVisite	défaut EN_ATTENTE
commentaire	TEXT	nullable
isArchived	boolean	default false
createdAt / updatedAt		

Statuts (StatutDemandeVisite): EN_ATTENTE → ACCEPTEE / REFUSEE / ANNULEE ; ACCEPTEE → AFFECTEE (Sprint 2) ; ACCEPTEE/AFFECTEE → REPLANIFICATION_DEMANDEE → (retour ACCEPTEE/AFFECTEE) ; ACCEPTEE/AFFECTEE/RAPPORT_REDIGE → CLOTUREE_SANS_SUITE / CLOTUREE_AVEC_CONTRAT. TERMINEE est un statut historique en lecture seule (non assignable via l'API).

RapportVisite (Sprint 2)

id	Long	PK
demandeVisite	→DemandeVisite	OneToOne, NOT NULL, UNIQUE
auteurAdmin	→User	NOT NULL
compteRendu	TEXT	nullable
documentUrl	String	nullable (PDF/image)
aboutie vers contrat)	boolean	default false – true = client intéressé (pipeline vers contrat)
createdAt	LocalDateTime	@PrePersist

Discussion + Message

Discussion:

- id, client →User (nullable), prospect →Prospect (nullable), annonce →Annonce
- guestToken String – référence de suivi pour le parcours invité
- messages →[]Message
- createdAt

Message:

- id, contenu TEXT, senderRole (CLIENT|ADMIN)
- isRead boolean, discussion →Discussion, createdAt

Contrat

id	Long	PK
client prospect)	→User	nullable – null avant conversion (contrat sur prospect)
prospect l'activation	→Prospect	nullable – exclusif avec client, converti à l'activation
annonce	→Annonce	NOT NULL
lead	→Lead	nullable
dateDebut	LocalDate	NOT NULL
dateFin	LocalDate	nullable
montant	BigDecimal	NOT NULL (precision 12, scale 2)
statut	StatutContrat	défaut EN_ATTENTE
typeContrat l'annonce	VENTE LOCATION	– déterminé par le typeTransaction de l'annonce
dureeLocationMois	Integer	nullable – obligatoire si typeContrat = LOCATION
dateFinGarantie	LocalDate	nullable – éligibilité des signalements (Sprint 4)
clausesContractuelles	TEXT	nullable
documentUrl	String	nullable
notes	TEXT	nullable
motifResiliation	TEXT	nullable
motifProlongation	TEXT	nullable
valideParSuperAdminAt	LocalDateTime	nullable – horodatage activation EN_ATTENTE → ACTIF
createdAt / updatedAt		

Statuts (StatutContrat) : EN_ATTENTE → ACTIF (active aussi la conversion prospect→client si besoin) → EXPIRE (job quotidien) ; ACTIF ↔ EN_ATTENTE_RESILIATION → RESILIE ; ACTIF ↔ PROLONGATION_EN_ATTENTE → ACTIF (nouvelle dateFin).

Signalement

id	Long	PK
contrat	→Contrat	NOT NULL
client	→User	NOT NULL
contenu	TEXT	NOT NULL
statut	StatutSignalement	défaut OUVERT
isRead	boolean	default false
reponseAdmin	TEXT	nullable
motifRejet (Sprint 4)	TEXT	nullable – obligatoire si REJETE/NON_ELIGIBLE
typeContratSnapshot	VENTE LOCATION	– type du contrat figé à la création (Sprint 4)
decisionAt	LocalDateTime	nullable – horodatage de la décision finale
createdAt	LocalDateTime	@PrePersist

Statuts (StatutSignalement) : OUVERT, EN_COURS, RESOLU, FERME (historiques) ; NON_ELIGIBLE (rejet auto : VENTE après fin de garantie), EN_ANALYSE (LOCATION éligible), ACCEPTE, REJETE (décisions finales, Sprint 4).

10. API REST — Endpoints

Toutes les routes sont préfixées par `/api/v1`.

La réponse standard est :

```
{
  "success": true,
  "message": "...",
  "data": { ... },
  "timestamp": "2025-01-01T12:00:00"
}
```

La réponse paginée est :

```
{
  "data": [...],
  "pagination": {
    "page": 0, "size": 10,
    "totalElements": 100, "totalPages": 10,
    "first": true, "last": false
  }
}
```

Auth — `/api/v1/auth`

Méthode	Route	Accès	Description
POST	<code>/register</code>	Public	Créer un compte CLIENT
POST	<code>/login</code>	Public	Connexion — retourne JWT
POST	<code>/logout</code>	Public	Invalider le token (blacklist)
GET	<code>/profile</code>	Authentifié	Profil de l'utilisateur courant
PUT	<code>/profile</code>	Authentifié	Modifier son profil
POST	<code>/change-password</code>	Authentifié	Changer son mot de passe (y compris 1re connexion forcée)
POST	<code>/admin</code>	SUPER_ADMIN	Créer un compte ADMIN
GET	<code>/admins</code>	SUPER_ADMIN	Lister les admins (paginé)
PUT	<code>/admins/{id}</code>	SUPER_ADMIN	Modifier un administrateur
PATCH	<code>/admins/{id}/archive</code>	SUPER_ADMIN	Archiver un admin
PATCH	<code>/admins/{id}/restore</code>	SUPER_ADMIN	Restaurer un admin

Méthode	Route	Accès	Description
PATCH	/admins/{id}/revoke	SUPER_ADMIN	Révoquer le rôle ADMIN (rétrograde en CLIENT)

Corps POST /register :

```
{
  "nomComplet": "Moussa Diallo",
  "email": "moussa@example.com",
  "telephone": "+221701234567",
  "motDePasse": "MonMotDePasse123!"
}
```

Réponse POST /login :

```
{
  "data": {
    "accessToken": "eyJhbGciOiJI...",
    "tokenType": "Bearer",
    "roles": ["CLIENT"]
  }
}
```

Annonces — /api/v1/annonces

Méthode	Route	Accès	Description
GET	/	Public	Lister les annonces actives (paginé)
GET	/ {id}	Public	Détail d'une annonce
POST	/search	Public	Recherche avancée (filtres combinables)
GET	/admin	ADMIN/SUPER_ADMIN	Toutes les annonces (paginé, y compris archivées)
GET	/admin/{id}	ADMIN/SUPER_ADMIN	Détail (vue admin, y compris archivée)
POST	/	ADMIN/SUPER_ADMIN	Créer une annonce
PUT	/ {id}	ADMIN/SUPER_ADMIN	Modifier une annonce
DELETE	/ {id}	ADMIN/SUPER_ADMIN	Archiver (soft delete)
PATCH	/ {id}/restore	ADMIN/SUPER_ADMIN	Restaurer une annonce archivée

Paramètres GET / (paginé) :

```
?page=0&size=10&sort=createdAt&direction=DESC
```

Corps POST /search :

```
{
  "libelle": "Villa",
  "typeBienId": 1,
  "prixMin": 50000000,
  "prixMax": 200000000,
  "departement": "Dakar",
  "quartier": "Almadies",
  "nbrPiecesMin": 3,
  "page": 0,
  "size": 9,
  "sortBy": "prix",
  "sortDir": "ASC"
}
```

Corps POST / (création) :

```
{
  "libelle": "Villa moderne à Almadies",
  "description": "...",
  "nbrPieces": 5,
  "surface": 250.0,
  "prix": 150000000,
  "adresse": "12 Rue des Ambassadeurs",
  "departement": "Dakar",
  "quartier": "Almadies",
  "typeBienId": 1,
  "typeTransaction": "VENTE",
  "commoditeIds": [1, 3, 5],
  "images": ["https://res.cloudinary.com/..."]
}
```

Référentiels

Types de bien — /api/v1/types-bien

Méthode	Route	Accès	Description
GET	/	Public	Types actifs (non paginé)
GET	/paged	Public	Types paginés (y compris archivés)
GET	/ {id}	Public	Détail
POST	/	ADMIN/SUPER_ADMIN	Créer un type
PUT	/ {id}	ADMIN/SUPER_ADMIN	Modifier
DELETE	/ {id}	ADMIN/SUPER_ADMIN	Archiver

Méthode	Route	Accès	Description
PATCH	<code>/{{id}}/restore</code>	ADMIN/SUPER_ADMIN	Restaurer

Commodités — `/api/v1/commodites`

Méthode	Route	Accès	Description
GET	<code>/</code>	Public	Commodités actives (non paginé)
GET	<code>/paged</code>	Public	Commodités paginées
GET	<code>/{{id}}</code>	Public	Détail
POST	<code>/</code>	ADMIN/SUPER_ADMIN	Créer
PUT	<code>/{{id}}</code>	ADMIN/SUPER_ADMIN	Modifier
DELETE	<code>/{{id}}</code>	ADMIN/SUPER_ADMIN	Archiver
PATCH	<code>/{{id}}/restore</code>	ADMIN/SUPER_ADMIN	Restaurer

Leads — `/api/v1/leads` (ADMIN/SUPER_ADMIN uniquement)

Méthode	Route	Description
POST	<code>/</code>	Créer un lead
GET	<code>/</code>	Lister (filtrable par <code>?statut=EN_COURS</code>)
GET	<code>/{{id}}</code>	Détail
PUT	<code>/{{id}}/status</code>	Modifier le statut (422 si lead lié à une visite — passe par la clôture de visite)
PUT	<code>/{{id}}/note</code>	Mettre à jour la note admin uniquement
GET	<code>/{{id}}/historique</code>	Historique des transitions (Audit Trail, paginé)

Corps POST / :

```
{
  "clientId": 42,
  "annonceId": 7,
  "visiteId": 3,
  "noteAdmin": "Client très motivé"
}
```

Corps PUT `/{{id}}/status` :

```
{
  "statut": "CONVERTI",
}
```

```
"noteAdmin": "Contrat signé"
}
```

Visites — /api/v1/visites

Méthode	Route	Accès	Description
POST	/	CLIENT	Créer une demande de visite
POST	/invite	Public	Créer une demande de visite (visiteur non authentifié) — crée/réutilise un Prospect, renvoie un prospectToken
GET	/suivi/{token}	Public	Suivre toutes les visites d'un prospect par son token
GET	/client	CLIENT	Mes visites (filtrable par statut)
GET	/admin	ADMIN/SUPER_ADMIN	Toutes les visites (filtrable par statut + typeTransaction)
GET	/_{id}	Authentifié	Détail (client : les siennes / admin : toutes)
PUT	/_{id}/status	CLIENT (annulation) / SUPER_ADMIN (accepter/refuser)	Changer le statut — l'acceptation/refus est réservée au SUPER_ADMIN
PUT	/_{id}/date	ADMIN/SUPER_ADMIN	Reprogrammer la date
PUT	/_{id}/modifier	CLIENT	Modifier sa demande (date/commentaire, si EN_ATTENTE)
PUT	/_{id}/cloture	ADMIN/SUPER_ADMIN	Clôturer une visite acceptée (SANS_SUITE ou AVEC_CONTRAT → crée le contrat)
DELETE	/_{id}	CLIENT	Annuler sa demande
GET	/_{id}/historique	ADMIN/SUPER_ADMIN	Historique des transitions (Audit Trail)
PUT	/_{id}/affecter	SUPER_ADMIN	Affecter un administrateur responsable (ACCEPTÉE → AFFECTÉE)
PUT	/_{id}/replanification	SUPER_ADMIN	Proposer une nouvelle date (→ REPLANIFICATION_DEMANDEE)
PUT	/_{id}/replanification/accepter	SUPER_ADMIN	Appliquer la date proposée

Méthode	Route	Accès	Description
POST	<code>/{{id}}/rapport</code>	ADMIN/SUPER_ADMIN	Rédiger le rapport de visite (→ RAPPORT_REDIGE)
GET	<code>/{{id}}/rapport</code>	ADMIN/SUPER_ADMIN	Consulter le rapport
POST	<code>/{{id}}/rapport/document</code>	ADMIN/SUPER_ADMIN	Téléverser le document du rapport (multipart, max 10 Mo)

Corps POST / (client authentifié) :

```
{
  "annonceId": 7,
  "dateVisite": "2025-03-15T10:00:00",
  "commentaire": "Je souhaite visiter samedi matin"
}
```

Corps POST /invite (visiteur non authentifié) :

```
{
  "annonceId": 7,
  "nom": "Diallo",
  "prenom": "Aminata",
  "telephone": "+221770000001",
  "email": "aminata@email.com",
  "adresse": "Almadies, Dakar",
  "dateVisite": "2025-03-15T10:00:00",
  "heureVisite": "10:00",
  "commentaire": "Disponible le matin"
}
```

Corps PUT /{{id}}/cloture (AVEC_CONTRAT) :

```
{
  "type": "AVEC_CONTRAT",
  "dureeLocationMois": 12
}
```

Le `typeContrat` (VENTE/LOCATION) n'est pas choisi manuellement : il est imposé par le `typeTransaction` de l'annonce. `dureeLocationMois` est obligatoire uniquement pour une annonce LOCATION.

Discussions & Messages — </api/v1/discussions>

Méthode	Route	Accès	Description
POST	/	CLIENT	Créer ou récupérer une discussion (idempotent)
POST	/invite	Public	Contacteur l'agence sans compte (visiteur)
GET	/token/{token}	Public	Lire une discussion invité par token
POST	/token/{token}/messages	Public	Répondre dans une discussion invité
GET	/client	CLIENT	Mes discussions (triées par dernière activité)
GET	/admin	ADMIN/SUPER_ADMIN	Toutes les discussions
GET	/id/messages	Authentifié	Messages (marque comme lus)
POST	/id/messages	Authentifié	Envoyer un message

Corps POST / :

```
{
  "annonceId": 7,
  "premierMessage": "Bonjour, je suis intéressé par ce bien."
}
```

Contrats — /api/v1/contrats

Méthode	Route	Accès	Description
POST	/	ADMIN/SUPER_ADMIN	Créer un contrat
GET	/client	CLIENT	Mes contrats (filtrable par statut)
GET	/admin	ADMIN/SUPER_ADMIN	Tous les contrats (filtrable)
GET	/id	Authentifié	Détail (client : les siens / admin : tous)
PUT	/id	ADMIN/SUPER_ADMIN	Modifier (statut, dates, montant, notes...) — l'activation déclenche la conversion prospect→client si besoin
PUT	/id/prospect	ADMIN/SUPER_ADMIN	Modifier les informations du prospect rattaché (avant activation)

Méthode	Route	Accès	Description
POST	<code>/{{id}}/document</code>	ADMIN/SUPER_ADMIN	Téléverser le document contractuel principal (multipart)
POST	<code>/{{id}}/documents</code>	ADMIN/SUPER_ADMIN	Ajouter des pièces jointes typées (URLs Cloudinary)
DELETE	<code>/{{id}}/documents/{{documentId}}</code>	ADMIN/SUPER_ADMIN	Supprimer une pièce jointe
PUT	<code>/{{id}}/resiliation</code>	CLIENT	Soumettre une demande de résiliation
PUT	<code>/{{id}}/resiliation/accepter</code>	ADMIN/SUPER_ADMIN	Accepter → statut RESILIE
PUT	<code>/{{id}}/resiliation/refuser</code>	ADMIN/SUPER_ADMIN	Refuser → statut ACTIF
PUT	<code>/{{id}}/prolongation</code>	CLIENT	Soumettre une demande de prolongation
PUT	<code>/{{id}}/prolongation/accepter</code>	ADMIN/SUPER_ADMIN	Accepter → nouvelle dateFin
PUT	<code>/{{id}}/prolongation/refuser</code>	ADMIN/SUPER_ADMIN	Refuser → statut ACTIF
GET	<code>/{{id}}/historique</code>	ADMIN/SUPER_ADMIN	Historique des transitions (Audit Trail)

Corps POST / (ADMIN) :

```
{
  "clientId": 42,
  "annonceId": 7,
  "leadId": 3,
  "dateDebut": "2025-02-01",
  "dateFin": "2026-01-31",
  "montant": 250000,
  "documentUrl": "https://...",
  "notes": "Contrat de location annuel"
}
```

Favoris — `/api/v1/favoris` (CLIENT uniquement)

Méthode	Route	Description
POST	<code>/{{annonceId}}/toggle</code>	Ajouter/retirer des favoris
GET	<code>/</code>	Mes favoris (paginé)
GET	<code>/ids</code>	Tous les IDs favoris (sans limite)
GET	<code>/{{annonceId}}/check</code>	Vérifier si une annonce est en favori

Signalements — </api/v1/signalements>

Méthode	Route	Accès	Description
POST	/	CLIENT	Signaler un problème sur un contrat (rejet auto si NON_ELIGIBLE)
GET	/client	CLIENT	Mes signalements
GET	/admin	ADMIN/SUPER_ADMIN	Tous les signalements (filtrable)
GET	/{"id"}	ADMIN/SUPER_ADMIN	Détail
PUT	/{"id"}/status	ADMIN/SUPER_ADMIN	Modifier statut + réponse/motif de rejet
PUT	/{"id"}/read	ADMIN/SUPER_ADMIN	Marquer comme lu
GET	/{"id"}/historique	ADMIN/SUPER_ADMIN	Historique des décisions (Audit Trail)

Dashboard — </api/v1/admin/dashboard> (ADMIN/SUPER_ADMIN)

Méthode	Route	Description
GET	/stats	Toutes les statistiques en un appel
GET	/activities	Activités récentes (paginé, filtrable)

Réponse /stats :

```
{
  "data": {
    "totalAnnonces": 45,
    "annoncesActives": 38,
    "totalClients": 120,
    "totalAdmins": 3,
    "totalContrats": 31,
    "contratsActifs": 18,
    "totalVisites": 89,
    "visitesEnAttente": 12,
    "visitesAujourd'hui": 2,
    "totalSignalements": 5,
    "signalementsOuverts": 2,
    "totalLeads": 67,
    "leadsEnCours": 23,
    "leadsConvertis": 31,
    "leadsAbandonnes": 13,
    "tauxConversionLeads": 70.45,
    "totalDiscussions": 93
  }
}
```

Audit Trail — Historique métier (ADMIN/SUPER_ADMIN)

Chaque module métier expose un endpoint de traçabilité enregistrant toutes les transitions de statut et actions effectuées.

Méthode	Route	Description
GET	/contrats/{id}/historique	Historique des transitions d'un contrat
GET	/leads/{id}/historique	Historique des transitions d'un lead
GET	/visites/{id}/historique	Historique des transitions et reprogrammations d'une visite
GET	/signalements/{id}/historique	Historique des décisions d'un signalement

Réponse paginée (structure commune) :

```
{
  "content": [
    {
      "id": 1,
      "ancienStatut": "EN_ATTENTE",
      "nouveauStatut": "ACTIF",
      "auteurId": 5,
      "auteurEmail": "admin@immosn.sn",
      "action": "ACTIVATION",
      "commentaire": null,
      "createdAt": "2024-02-01T10:00:00"
    }
  ],
  "page": 0, "size": 20, "totalElements": 3, "totalPages": 1, "last": true
}
```

VisiteHistoryDto inclut en plus ancienneDateVisite et nouvelleDateVisite pour tracer les reprogrammations.

Géolocalisation — /api/v1/locations (Public)

Méthode	Route	Description
GET	/departements	Liste des départements disponibles
GET	/quartiers?departement={nom}	Quartiers d'un département
GET	/geocode? departement=&quartier=&adresse=	Coordonnées GPS via Nominatim (prévisualisation carte)

Départements disponibles (région Dakar uniquement) :

- Dakar, Pikine, Guédiawaye, Rufisque, Keur Massar

11. Sécurité et authentification

Flux d'authentification JWT

1. POST /auth/login { email, motDePasse }
 - ↳ Spring Security valide les credentials (BCrypt)
 - ↳ JwtTokenProvider génère un token HMAC-SHA512 (24h)
 - ↳ Retourne { accessToken, tokenType: "Bearer" }
2. Requêtes authentifiées :
 - ↳ Header : Authorization: Bearer <token>
 - ↳ JwtAuthenticationFilter extrait et valide le token
 - ↳ Vérifie que le token n'est pas blacklisté
 - ↳ Injecte l'utilisateur dans le SecurityContext
3. POST /auth/logout
 - ↳ Ajoute le token à la table BlacklistedToken
 - ↳ Token invalide pour toutes les requêtes suivantes

Routes publiques explicites (sans JWT)

Toutes les routes `/api/v1/visites/**`, `/api/v1/discussions/**`, `/api/v1/contrats/**`, `/api/v1/signalements/**`, `/api/v1/favoris/**` sont `authenticated()` **par défaut**, à l'exception des routes suivantes, explicitement publiques pour le parcours visiteur :

- POST /api/v1/visites/invite
- GET /api/v1/visites/suivi/**
- POST /api/v1/discussions/invite
- GET /api/v1/discussions/token/**
- POST /api/v1/discussions/token/**

RBAC — Matrice des permissions

Ressource	Public / Prospect	CLIENT	ADMIN	SUPER_ADMIN
Annonces (lecture)	✓	✓	✓	✓
Annonces (écriture)	✗	✗	✓	✓
Visite — créer (invité)	✓	—	—	—
Visite — créer (compte)	✗	✓	✗	✗
Visite — accepter/refuser	✗	✗	✗	✓
Visite — affecter/replanifier	✗	✗	✗	✓
Visite — rapport/clôture	✗	✗	✓	✓
Discussion — contacter (invité)	✓	—	—	—
Leads	✗	✗	✓	✓
Contrats	✗	✓ (les siens)	✓ (tous)	✓

Ressource	Public / Prospect	CLIENT	ADMIN	SUPER_ADMIN
Favoris	✗	✓	✗	✓
Signalements	✗	✓ (les siens)	✓ (tous)	✓
Dashboard	✗	✗	✓	✓
Gestion admins	✗	✗	✗	✓
/locations/geocode	✓	✓	✓	✓

Génération de la clé JWT

```
openssl rand -base64 64
# Coller la valeur dans JWT_SECRET du fichier .env
```

12. Flux métier

Cycle de vie d'un prospect → lead → contrat (visiteur non authentifié)

```
VISITEUR consulte une annonce (sans compte)
├── Contacte l'agence : POST /discussions/invite
│   └── Discussion + Message rattachés à un Prospect (créé/réutilisé par
│       email)
├── Demande une visite : POST /visites/invite
│   ├── Prospect créé/réutilisé, token de suivi UUID retourné
│   │   (prospectToken)
│   ├── Notification email + SMS du numéro de suivi (voir §13)
│   │   statut: EN_ATTENTE – Lead auto-crée (prospect + annonce + visite)
│   ├── SUPER_ADMIN accepte → ACCEPTEE
│   ├── SUPER_ADMIN affecte un admin → AFFECTEE
│   ├── Admin rédige un rapport (visite effectuée) → RAPPORT_REDIGE
│   └── Admin clôture :
│       ├── SANS_SUITE → Lead: ABANDONNE
│       └── AVEC_CONTRAT → Contrat créé (statut EN_ATTENTE, rattaché
│           au Prospect)
│
│                               Lead: CONVERTI
```

Conversion prospect → client

```
ADMIN active un contrat : PUT /contrats/{id} { statut: "ACTIF" }
├── Si le contrat n'a pas de client mais un prospect :
│   └── ProspectConversionService.convertirProspect(prospectId)
```

```

    ↳ Idempotent : prospect déjà converti → renvoie le compte
existant
    ↳ Refuse si un compte existe déjà pour cet email (409)
    ↳ Crée un User CLIENT :
      compteGenereAuto=true, motDePasseAChanger=true
      mot de passe temporaire généré (14 car., BCrypt)
      rôle CLIENT assigné
    ↳ Rattache au nouveau compte les Discussion et
DemandeVisite du prospect
    ↳ prospect.convertedUser = nouveauClient
    ↳ Notifications (voir §13) :
      ↳ Email + SMS au client (identifiants de
connexion)
      ↳ Email à chaque SUPER_ADMIN (nouveau compte
créé)
      ↳ Contrat.client = nouveauClient ; valideParSuperAdminAt = now()
    ↳ Le client doit changer son mot de passe temporaire à la 1re connexion
      (page dédiée frontend – flag motDePasseAChanger)

```

Cycle de vie d'un contrat

```

ADMIN crée Contrat → statut: EN_ATTENTE
|
↳ ADMIN active → statut: ACTIF
  | (déclenche la conversion prospect→client si le contrat n'a pas
  | encore de client)
  |
  ↳ [Cron quotidien 00h05] dateFin dépassée → EXPIRE
  |
  ↳ CLIENT demande résiliation
    ↳ EN_ATTENTE_RESILIATION → [ADMIN] RESILIE
  |
  ↳ CLIENT demande prolongation
    ↳ PROLONGATION_EN_ATTENTE → [ADMIN] ACTIF (nouvelle
dateFin)

```

Flux de messagerie

```

CLIENT (ou visiteur) ouvre une annonce → clique "Contacter"
|
↳ POST /discussions { annonceId, premierMessage } (client)
  POST /discussions/invite { ... } (visiteur, public)
  | Idempotent: même (client|prospect, annonce) → même discussion
  ↳ ADMIN voit la discussion dans le panel admin
    ↳ Répond → Message(senderRole: ADMIN)
      ↳ Le demandeur reçoit la réponse (isRead: false)
        ↳ Marque comme lu à l'ouverture

```

Géocodage des annonces

```

ADMIN remplit formulaire → sélectionne département + quartier
├── Frontend appelle GET /locations/geocode (prévisualisation carte)
│   └── Backend appelle Nominatim (OpenStreetMap)
│       └── Retourne latitude/longitude pour aperçu
├── ADMIN soumet l'annonce (POST /annonces)
│   └── Backend régéocode automatiquement (source de vérité)
│       └── Coordonnées sauvegardées en base
│           └── Carte affichée sur la page de détail

```

13. Notifications email / SMS

Le module `shared/service/` centralise l'envoi de notifications. Chaque canal (email, SMS) échoue **indépendamment et silencieusement** (try/catch, log seulement) : une panne SMTP ou Twilio ne bloque jamais la création d'une visite, d'une discussion ou la conversion d'un prospect.

Services

Service	Déclencheur	Contenu
<code>NotificationService</code> (impl. <code>ClientWelcomeNotificationServiceImpl</code>)	Conversion prospect → client (activation contrat)	Email + SMS au client (identifiants de connexion) ; email à chaque <code>SUPER_ADMIN</code> (nouveau compte créé)
<code>VisiteTrackingNotificationService</code> (impl. <code>VisiteTrackingNotificationServiceImpl</code>)	<code>POST</code> <code>/visites/invite</code>	Email + SMS au prospect (numéro de suivi)

Activation

Contrôlée par `MAIL_ENABLED` / `TWILIO_ENABLED` (voir §7). Si désactivé ou en erreur, le canal correspondant est simplement journalisé (`INFO` tentative, `ERROR` échec — jamais de mot de passe ou de token Twilio en clair dans les logs).

Limitation connue — compte Twilio Trial

Un compte Twilio en mode **Trial** ne peut envoyer un SMS qu'à un numéro explicitement **vérifié** dans la console Twilio (*Phone Numbers* → *Manage* → *Verified Caller IDs*). Toute tentative vers un numéro non vérifié échoue avec l'erreur `unverified number` (capturée et journalisée, sans impact sur le flux métier). Passer le compte Twilio en payant lève cette restriction — nécessaire avant un lancement en production avec de vrais clients.

14. Frontend — Vue.js

Stores Pinia

authStore

```
// État
token          // JWT depuis localStorage
user           // { id, nomComplet, email, roles, ... }
role           // 'CLIENT' | 'ADMIN' | 'SUPER_ADMIN'

// Getters
isAuthenticated
isAdmin        // role === ADMIN || SUPER_ADMIN

// Actions
login(email, motDePasse) // POST /auth/login → set token + redirect
logout()                // POST /auth/logout → clear + redirect /connexion
fetchProfile()          // GET /auth/profile → refresh user data
init()                  // Appelé au montage App.vue pour restaurer la session
```

favorisStore

```
// État
favorisIds      // Set<Long> – chargé une fois par session

// Actions
loadFavoris()    // GET /favoris/ids → remplit le Set
toggle(annonceId) // Optimiste : toggle immédiat + rollback si erreur
isFavori(annonceId) // Boolean – accès synchrone
reset()          // Vidé à la déconnexion
```

Intercepteurs Axios ([services/api.js](#))

Requête — Ajoute automatiquement le token JWT :

```
headers.Authorization = `Bearer ${token}`
```

Réponse — Gestion centralisée des erreurs :

- **401** → clear localStorage + redirect [/connexion](#)
- Enrichit les erreurs avec [userMessage](#) pour l'affichage UI

Guards de navigation (Vue Router)

```
// Pour chaque route avec meta.requiresAuth = true
if (!isAuthenticated) → redirect /connexion

// Pour les routes admin (meta.role = 'ADMIN')
if (!isAdmin) → redirect /annonces
```

```
// Compte avec mot de passe temporaire (motDePasseAChanger = true)
if (mustChangePassword) → redirect /first-login (forcé, jusqu'au changement)
```

Parcours visiteur (prospect)

- **/visite-demande** — page dédiée (hors modal) pour la demande de visite sans compte : identité, adresse, créneau souhaité, message. Réutilise **PhoneInput** et affiche le numéro de suivi (**prospectToken**) après succès.
- **/suivi-visite?token=...** — suit toutes les demandes de visite liées à un token de prospect.
- Le contact agence sans compte (**/discussions/invite**) reste accessible depuis la page détail d'annonce.

Pattern de service (exemple)

```
// Appel
const res = await annonceService.getAll(0, 10)
// res.data.data → tableau d'annonces
// res.data.pagination → { page, size, totalElements, totalPages }

// Toutes les réponses suivent RestResponse<T> ou PagedResponse<T>
```

15. Géolocalisation

Architecture

```
Vue (formulaire) → GET /api/v1/locations/geocode (ADMIN auth)
                  ↳ GeoCodingService → Nominatim OpenStreetMap
                  ↳ Retourne { latitude, longitude } (prévisualisation)

Soumission formulaire → Spring Boot régéocode automatiquement
                      ↳ Sauvegarde les coordonnées définitives en base
```

Données disponibles (région Dakar)

Département	Exemples de quartiers
Dakar	Almadies, Plateau, Médina, Yoff, Ngor, Ouakam, Mermoz, Point E...
Pikine	Pikine Est, Pikine Ouest, Thiaroye, Yeumbeul, Diamaguène...
Guédiawaye	Golf Sud, Hamo, Sam Notaire, Wakhinane Nimzatt...
Rufisque	Rufisque Est/Nord/Ouest, Bargny, Diamniadio, Sébikotane
Keur Massar	Keur Massar Nord/Sud, Malika, Jaxaay-Parcelles...

Composant `LocationMap.vue`

```
<!-- Lecture seule -->
<LocationMap :latitude="annonce.latitude" :longitude="annonce.longitude"
height="260px" />

<!-- Draggable (formulaire admin) -->
<LocationMap
  :latitude="form.latitude"
  :longitude="form.longitude"
  :draggable="true"
  height="280px"
  @update:latitude="v => form.latitude = v"
  @update:longitude="v => form.longitude = v"
/>
```

16. Déploiement Docker

Fichiers disponibles

Fichier	Usage	Base de données
<code>docker-compose.yml</code>	Production — VPS (nginx.vps.conf, profil Spring <code>neon</code>)	PostgreSQL Neon
<code>docker-compose.dev.yml</code>	Développement local	H2 in-memory

Commandes utiles

```
# Développement
docker compose -f docker-compose.dev.yml up --build

# Production
docker compose up -d --build

# Logs backend
docker compose logs -f backend

# Logs nginx
docker compose logs -f nginx

# Accéder au shell du backend
docker compose exec backend sh

# Rebuild un seul service
docker compose up --build backend -d
```

Health check

Le backend expose **GET /actuator/health** (sans requête DB) — réponse en < 1 s même après cold start Neon.

Docker Compose attend que le backend soit **healthy** avant de démarrer Nginx.

```
healthcheck:
  test: ["CMD-SHELL", "wget -q -O /dev/null http://127.0.0.1:8080/actuator/health
|| exit 1"]
  interval: 30s
  timeout: 10s
  retries: 5
  start_period: 90s
```

17. Configuration Nginx

Rôles de Nginx

1. **Serveur de fichiers statiques** — Sert le build Vite (/app/dist)
2. **SPA fallback** — Toutes les routes inconnues → **index.html** (Vue Router)
3. **Reverse proxy** — **/api/*** → **http://backend:8080**
4. **Cache assets** — 1 an pour les fichiers hachés Vite (.js, .css, .woff2)
5. **Headers de sécurité** — **X-Frame-Options**, **X-Content-Type-Options**, **X-XSS-Protection**
6. **Compression** — gzip activé (min. 1 Ko, niveau 5)
7. **H2 Console bloquée** — **/h2-console** renvoie 403 en production

Configuration type

```
location /api/ {
  proxy_pass http://backend:8080;
  proxy_set_header Host $host;
  proxy_set_header X-Real-IP $remote_addr;
}

location / {
  root /usr/share/nginx/html;
  try_files $uri /index.html; # SPA fallback
}

location ~* \.(js|css|woff2|png|jpg|svg)$ {
  expires 1y;
  add_header Cache-Control "public, immutable";
}
```

Codes d'erreur HTTP

Code	Cas
200	Succès
201	Ressource créée
204	Succès sans contenu (archive, restore, annulation)
400	Paramètre invalide / validation échouée
401	Token absent ou invalide
403	Rôle insuffisant
404	Ressource non trouvée
409	Conflit (email déjà utilisé)
422	Règle métier violée (ex : archiver un contrat actif, modifier le statut d'un lead lié à une visite)
500	Erreur interne non gérée

Conventions de code

Backend

- **Packages** : `sn.immosn.backend.<domaine>.<couche>`
- **DTOs** : `{Entité}CreateRequestDto`, `{Entité}UpdateRequestDto`, `{Entité}ResponseDto`
- **Services** : interface + `Impl` séparés
- **Réponses** : toujours wrappées dans `RestResponse<T>` ou `PagedResponse<T>`
- **Exceptions** : `EntityNotFoundException` (404), `EntityExistException` (409), `IllegalStateException` (422)
- **Soft delete** : champ `isArchived` sur toutes les entités supprimables
- **Transactions** : `@Transactional` sur les méthodes de service mutantes
- **Notifications** : try/catch par canal (email/SMS), jamais de rethrow — l'échec d'une notification ne bloque jamais un flux métier

Frontend

- **Composants** : `<script setup>` (Composition API)
- **Services** : une fonction par endpoint, retournent la Promise Axios
- **Stores** : Pinia avec `defineStore`, nommés `use{Nom}Store`
- **Vues admin** : suffixe `AdminView` ou préfixe `Admin`
- **Gestion d'erreurs** : `err.userMessage || err.response?.data?.message` pour l'affichage toast
- **Mot de passe** : utiliser `InputField` (toggle afficher/masquer intégré) plutôt qu'un `<input type="password">` brut